

Basic Information

Experiment No: 06

Date: 23/09/2026

Title: Java Collection Framework using Collection Interface and ArrayList

Objective / Aim

To implement the Java Collection Framework using the "Collection" interface and "ArrayList", and demonstrate commonly used collection methods such as "add()", "addAll()", "contains()", "remove()", "size()", "iterator()", and "clear()".

System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

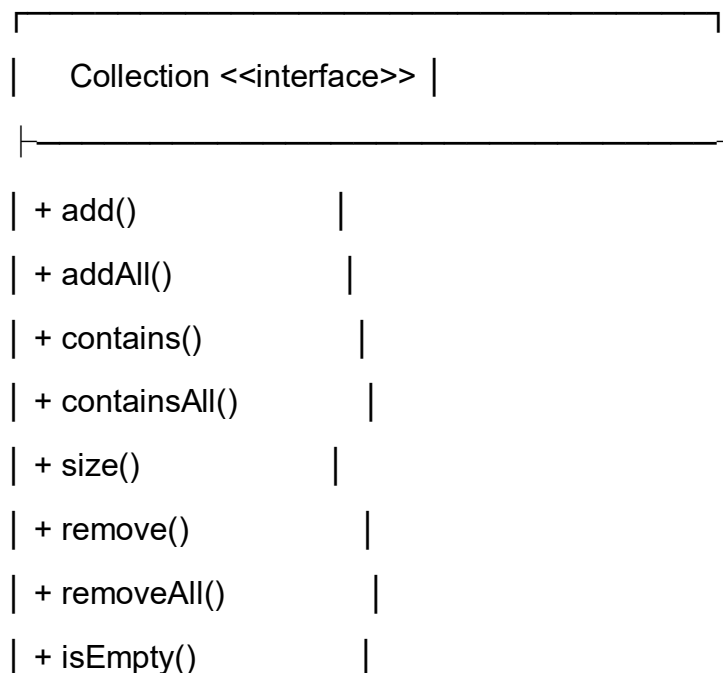
Software

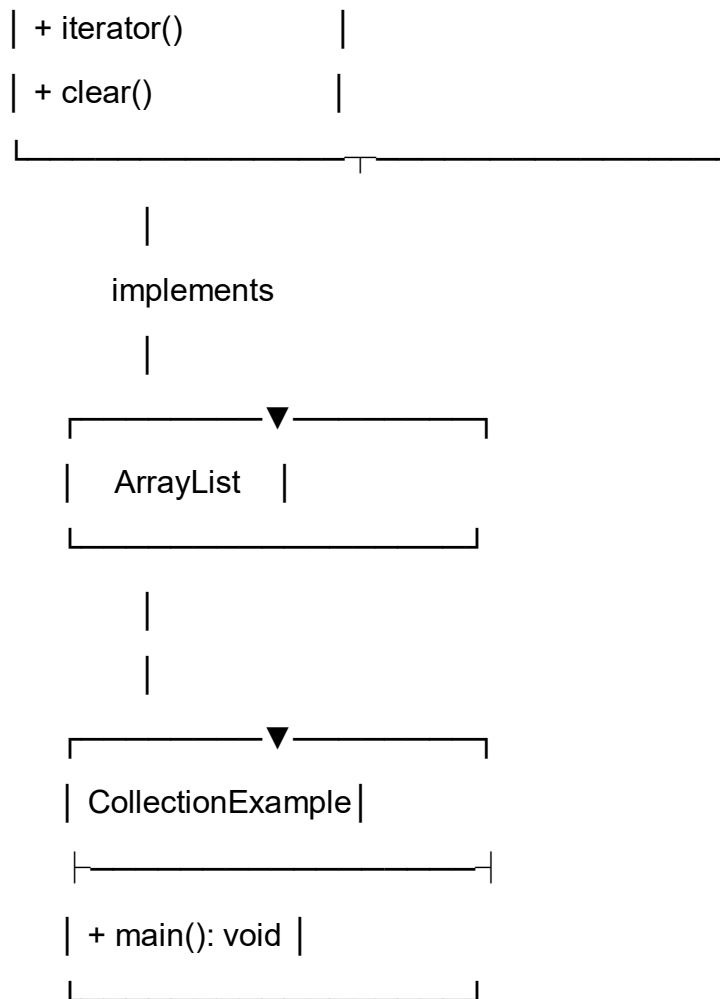
- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

Algorithm / Logic Description

1. Create a "Collection" using "ArrayList".
2. Add elements using "add()".
3. Create another collection and add it using "addAll()".
4. Check elements using "contains()" and "containsAll()".
5. Find the collection size using "size()".
6. Remove elements using "remove()" and "removeAll()".
7. Check whether the collection is empty using "isEmpty()".
8. Use "Iterator" to display the elements.
9. Remove all elements using "clear()".
10. End the program.

UML Class Diagram





Relationship: "ArrayList" implements the "Collection" interface. The program uses an "Iterator" to traverse the collection elements.

Source Code

```
package javacore;

import java.util.Collection;
import java.util.ArrayList;
import java.util.Iterator;

public class CollectionExample {
```

```
public static void main(String[] args) {

    // Create Collection using ArrayList
    Collection<String> c = new ArrayList<>();

    // add()
    c.add("Apple");
    c.add("Banana");
    c.add("Mango");

    System.out.println("Collection: " + c);

    // addAll()
    Collection<String> c2 = new ArrayList<>();

    c2.add("Orange");
    c2.add("Grapes");

    c.addAll(c2);

    System.out.println("After addAll: " + c);

    // contains()
    System.out.println(
        "Contains Apple: " + c.contains("Apple")
    );

    // containsAll()
```

```
System.out.println(  
    "Contains all c2: " + c.containsAll(c2)  
);
```

```
// size()  
System.out.println(  
    "Size: " + c.size()  
);
```

```
// remove()  
c.remove("Banana");
```

```
System.out.println(  
    "After remove: " + c  
);
```

```
// removeAll()  
c.removeAll(c2);
```

```
System.out.println(  
    "After removeAll: " + c  
);
```

```
// isEmpty()  
System.out.println(  
    "Is Empty: " + c.isEmpty()  
);
```

```
// iterator()
```

```

    Iterator<String> it = c.iterator();

    System.out.println("Using Iterator:");

    while (it.hasNext()) {
        System.out.println(it.next());
    }

    // clear()
    c.clear();

    System.out.println(
        "After clear: " + c
    );
}
}

---
```

Compilation & Execution Commands

Save the source code as:

CollectionExample.java

If you are using the "package javacore;" statement, save the file inside the appropriate "javacore" package/folder.

Compile the program:

```
javac CollectionExample.java
```

Run the program:

java CollectionExample

Output Screen

Collection: [Apple, Banana, Mango]

After addAll: [Apple, Banana, Mango, Orange, Grapes]

Contains Apple: true

Contains all c2: true

Size: 5

After remove: [Apple, Mango, Orange, Grapes]

After removeAll: [Apple, Mango]

Is Empty: false

Using Iterator:

Apple

Mango

After clear: []

Result

The Java Collection Framework was successfully implemented using the "Collection" interface and "ArrayList". The program demonstrates important collection operations including adding, searching, removing, checking size, iterating, and clearing elements.